Cours Python

Fonctions · POO · Structures de données

Document source RAG — PyTutor

01 Les fonctions

Définition, paramètres, lambda, décorateurs

1.1 Définir une fonction

En Python, une fonction se déclare avec le mot-clé **def**, suivi du nom, des paramètres entre parenthèses et d'un bloc indenté. La docstring (premier littéral de chaîne) documente la fonction et est accessible via **help()**.

```
def saluer(nom):
"""Retourne un message de bienvenue."""
return f"Bonjour, {nom} !"

print(saluer("Alice")) # Bonjour, Alice !
print(saluer.__doc__) # Retourne un message de bienvenue.
```

1.2 Types de paramètres

Python offre une grande flexibilité dans la déclaration des paramètres. Les paramètres positionnels sont obligatoires. Les paramètres avec valeur par défaut sont optionnels. ***args** capture un nombre variable d'arguments positionnels dans un tuple, et ****kwargs** capture les arguments nommés dans un dictionnaire.

```
# Paramètre avec valeur par défaut
def connexion(host, port=8080):
print(f"Connexion a {host}:{port}")

# *args — tuple d'arguments positionnels
def somme(*args):
return sum(args)

print(somme(1, 2, 3, 4)) # 10

# **kwargs — dict d'arguments nommés
def afficher(**kwargs):
for cle, val in kwargs.items():
print(f"{cle} = {val}")

afficher(langue="fr", niveau="debutant")
```

Syntaxe	Description
`def f(a, b)`	Positionnels — obligatoires, ordre fixe
`def f(a, b=10)`	Valeur par défaut — b optionnel
`def f(*args)`	Tuple de N arguments positionnels
`def f(**kwargs)`	Dict d'arguments nommés
`def f(a, /, b, *, c)`	Positional-only / keyword-only (Python 3.8+)

1.3 Fonctions lambda

Une fonction **lambda** est une fonction anonyme définie en une seule expression. Elle est souvent utilisée comme argument d'une fonction d'ordre supérieur (map, filter, sorted). Elle ne peut contenir qu'une expression et ne supporte pas les instructions (if/else sur plusieurs lignes, boucles).

```
carre = lambda x: x ** 2
print(carre(5)) # 25

# Tri par longueur de chaîne
mots = ['banane', 'kiwi', 'ananas', 'figue']
tries = sorted(mots, key=lambda m: len(m))
print(tries) # ['kiwi', 'figue', 'banane', 'ananas']

# Filtrer les nombres pairs
pairs = list(filter(lambda x: x % 2 == 0, range(10)))
print(pairs) # [0, 2, 4, 6, 8]
```

Attention : Les lambdas ne remplacent pas les fonctions nommées pour du code complexe. Préfère def quand la logique dépasse une ligne.

1.4 Décorateurs

Un **décorateur** est une fonction qui prend une fonction en entrée, l'enveloppe dans une autre fonction (wrapper), et retourne le wrapper. La syntaxe **@nom_decorateur** au-dessus d'une définition est un raccourci syntaxique pour *func = decorateur(func)*.

```
import time

def timer(func):
"""Mesure le temps d'execution d'une fonction."""
def wrapper(*args, **kwargs):
debut = time.time()
resultat = func(*args, **kwargs)
duree = time.time() - debut
print(f"{func.__name__} : {duree:.4f}s")
return resultat
return wrapper

@timer
def calcul_lourd(n):
return sum(i**2 for i in range(n))

calcul_lourd(1_000_000) # calcul_lourd : 0.1523s
```

Pour conserver les métadonnées de la fonction originale (nom, docstring), utilise **@functools.wraps(func)** dans le wrapper :

```
from functools import wraps

def mon_deco(func):
@wraps(func) # conserve __name__, __doc__
def wrapper(*args, **kwargs):
print('Avant...')
return func(*args, **kwargs)
return wrapper
```

Astuce : *Les décorateurs sont la base des frameworks Python : @app.route dans Flask, @property, @staticmethod, @classmethod sont tous des décorateurs.*

02 POO — Programmation Orientée Objet

Classes, héritage, méthodes spéciales

2.1 Définir une classe

Une **classe** est un modèle qui regroupe des **attributs** (données) et des **méthodes** (fonctions). La méthode **__init__** est le constructeur — elle est appelée automatiquement lors de la création d'une instance. Le premier paramètre **self** représente l'instance courante.

```
class Chien:
"""Représente un chien domestique."""
espece = 'Canis lupus familiaris' # attribut de classe

def __init__(self, nom, race, age):
self.nom = nom # attribut d'instance
self.race = race
self.age = age

def aboyer(self):
return f"{self.nom} dit : Wouf !"

def anniversaire(self):
self.age += 1
return f"{self.nom} a maintenant {self.age} ans"

rex = Chien("Rex", "Berger Allemand", 3)
print(rex.aboyer()) # Rex dit : Wouf !
print(Chien.espece) # Canis lupus familiaris
```

2.2 Héritage

L'**héritage** permet à une classe enfant de réutiliser et d'étendre le comportement d'une classe parente. La fonction **super()** appelle les méthodes de la classe parente. Python supporte l'héritage multiple.

```
class Animal:
def __init__(self, nom, age):
self.nom = nom
self.age = age

def description(self):
return f"{self.nom}, {self.age} ans"

class Chat(Animal): # Chat hérite d'Animal
def __init__(self, nom, age, indoor):
super().__init__(nom, age) # appel du parent
self.indoor = indoor

def ronronner(self):
return f"{self.nom} ronronne..."
```

```
def description(self): # surcharge (override)
base = super().description()
lieu = "intérieur" if self.indoor else "extérieur"
return f"{base} — chat d'{lieu}"

mimi = Chat("Mimi", 4, indoor=True)
print(mimi.description())
print(isinstance(mimi, Animal)) # True
```

2.3 Méthodes spéciales (dunder)

Les méthodes dont le nom est entouré de doubles underscores (__**méthode**__) sont appelées méthodes spéciales ou *dunder methods*. Elles permettent de personnaliser le comportement des objets avec les opérateurs Python.

```
class Vecteur:
def __init__(self, x, y):
self.x, self.y = x, y

def __repr__(self):
return f"Vecteur({self.x}, {self.y})"

def __add__(self, other):
return Vecteur(self.x + other.x, self.y + other.y)

def __len__(self):
return 2

def __eq__(self, other):
return self.x == other.x and self.y == other.y

v1 = Vecteur(1, 2)
v2 = Vecteur(3, 4)
print(v1 + v2) # Vecteur(4, 6)
print(len(v1)) # 2
```

Méthode	Déclenchée par
`__init__(self, ...)`	Constructeur
`__repr__(self)`	Représentation officielle (debug)
`__str__(self)`	Représentation lisible (print)
`__len__(self)`	len(obj)
`__add__(self, other)`	obj1 + obj2
`__eq__(self, other)`	obj1 == obj2
`__lt__(self, other)`	obj1 < obj2
`__getitem__(self, key)`	obj[key]
`__iter__(self)`	for item in obj

Méthode	Déclenchée par
`__enter__ / __exit__`	with obj as x

2.4 Propriétés et encapsulation

Le décorateur **@property** transforme une méthode en attribut calculé. Les attributs préfixés par _ sont conventionnellement privés, ceux avec __ déclenchent le name mangling (protection forte).

```
class Temperature:
def __init__(self, celsius):
self._celsius = celsius

@property
def celsius(self):
return self._celsius

@celsius.setter
def celsius(self, valeur):
if valeur < -273.15:
raise ValueError('Temperature sous le zero absolu')
self._celsius = valeur

@property
def fahrenheit(self): # attribut calculé en lecture seule
return self._celsius * 9/5 + 32

t = Temperature(100)
print(t.fahrenheit) # 212.0
t.celsius = 37 # utilise le setter
```

03 Structures de données

Listes, tuples, dictionnaires, sets, comprehensions

3.1 Listes

La **liste** est la structure de données la plus utilisée en Python. Elle est *mutable*, *ordonnée* et accepte des types mixtes. Les indices commencent à 0 ; les indices négatifs comptent depuis la fin.

```
fruits = ['pomme', 'banane', 'cerise']

# Accès et slicing
print(fruits[0]) # 'pomme'
print(fruits[-1]) # 'cerise'
print(fruits[1:3]) # ['banane', 'cerise']
print(fruits[::-1]) # ['cerise', 'banane', 'pomme'] (inverse)

# Modification
fruits.append('mangue') # ajout fin
fruits.insert(1, 'abricot') # ajout à l'index 1
fruits.remove('banane') # supprime par valeur
del fruits[0] # supprime par index
popped = fruits.pop() # retire et retourne le dernier

# Méthodes utiles
fruits.sort() # tri en place
copie = fruits.copy() # copie superficielle
print(fruits.count('pomme')) # nb occurrences
print(fruits.index('cerise')) # premier index trouvé
```

Note : *list.sort() modifie la liste en place et retourne None. sorted(liste) retourne une nouvelle liste triée sans modifier l'originale.*

3.2 Dictionnaires

Le **dictionnaire** stocke des paires clé-valeur. Depuis Python 3.7, il conserve l'ordre d'insertion. Les clés doivent être *hashables* (immuables : str, int, tuple). L'accès est en O(1) en moyenne.

```
etudiant = {
"nom": "Alice",
"age": 22,
"notes": [14, 17, 12]
}

# Accès
print(etudiant["nom"]) # Alice
print(etudiant.get("ville", "?")) # ? (valeur par défaut)

# Modification
etudiant["email"] = "alice@edu.fr" # ajout
etudiant["age"] = 23 # mise à jour
del etudiant["notes"] # suppression
```

```
# Itération
for cle, val in etudiant.items():
print(f"{cle}: {val}")

# Méthodes
cles = list(etudiant.keys())
valeurs = list(etudiant.values())
etudiant.update({"ville": "Paris", "age": 24})
```

3.3 Tuples et sets

Le **tuple** est une liste *immuable* — idéal pour des données qui ne doivent pas changer (coordonnées, clés composées). Le **set** est une collection non ordonnée de valeurs uniques, optimisé pour les tests d'appartenance (O(1)) et les opérations ensemblistes.

```
# Tuple
point = (3.0, 4.0)
x, y = point # déballage (unpacking)
print(x) # 3.0
# point[0] = 5 => TypeError !

# Set
couleurs = {'rouge', 'vert', 'bleu', 'rouge'}
print(couleurs) # {'rouge', 'vert', 'bleu'} (doublon éliminé)

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b) # union : {1, 2, 3, 4, 5, 6}
print(a & b) # intersection : {3, 4}
print(a - b) # différence : {1, 2}
print(a ^ b) # différence symétrique : {1, 2, 5, 6}
```

3.4 Compréhensions

Les **compréhensions** offrent une syntaxe concise pour construire des collections à partir d'une expression et d'une boucle. Il existe des compréhensions pour les listes, dictionnaires, sets et des expressions génératrices (paresseuses, économes en mémoire).

```
# List comprehension
carres = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Avec filtre
pairs = [x for x in range(20) if x % 2 == 0]

# Dict comprehension
mots = ['chat', 'chien', 'lapin']
longueurs = {m: len(m) for m in mots}
# {'chat': 4, 'chien': 5, 'lapin': 5}

# Set comprehension
initiales = {m[0].upper() for m in mots}
# {'C', 'L'}
```

```
# Expression génératrice (économe en mémoire)
total = sum(x**2 for x in range(1_000_000))

# Compréhension imbriquée (matrice aplatie)
matrice = [[1,2,3],[4,5,6],[7,8,9]]
plat = [n for ligne in matrice for n in ligne]
```

Astuce : *Préfère les expressions génératrices (parenthèses) aux list comprehensions (crochets) quand tu n'as besoin de la liste qu'une seule fois — elles ne chargent pas tout en mémoire.*

3.5 Tableau comparatif

Structure	Mutable	Ordonné	Doublons	Usage typique
list []	Oui	Oui	Oui	Séquences modifiables
tuple ()	Non	Oui	Oui	Données immuables, clés
dict {}	Oui	Oui	Clés non	Associations clé-valeur
set {}	Oui	Non	Non	Unicité, opérations ensemble

04 Gestion des erreurs

Exceptions, try/except, context managers

4.1 Exceptions intégrées

Python utilise les **exceptions** pour signaler les erreurs d'exécution. Toutes héritent de **BaseException**. Les plus courantes héritent d'**Exception**. La hiérarchie permet de capturer des familles d'erreurs avec un seul except.

Exception	Cause fréquente
`ValueError`	Valeur incorrecte (int('abc'))
`TypeError`	Mauvais type (1 + '2')
`KeyError`	Clé absente dans un dict
`IndexError`	Indice hors limites dans une liste
`AttributeError`	Attribut ou méthode inexistant
`FileNotFoundError`	Fichier introuvable
`ZeroDivisionError`	Division par zéro
`ImportError`	Module introuvable
`StopIteration`	Fin d'un itérateur
`RuntimeError`	Erreur générique d'exécution

4.2 try / except / else / finally

```
def diviser(a, b):
try:
resultat = a / b
except ZeroDivisionError:
print('Erreur : division par zero')
return None
except TypeError as e:
print(f"Type invalide : {e}")
return None
else:
# Exécuté SEULEMENT si aucune exception
print(f"Succès : {resultat}")
return resultat
finally:
# Toujours exécuté (nettoyage)
print('Fin de la division')

diviser(10, 2) # Succès : 5.0
diviser(10, 0) # Erreur : division par zero
```

4.3 Exceptions personnalisées

```
class ErreurMetier(Exception):
"""Base pour les erreurs métier de l'application."""
pass

class SoldeInsuffisant(ErreurMetier):
def __init__(self, solde, montant):
self.solde = solde
self.montant = montant
super().__init__(
f"Solde {solde}€ insuffisant pour retirer {montant}€"
)

def retirer(solde, montant):
if montant > solde:
raise SoldeInsuffisant(solde, montant)
return solde - montant

try:
retirer(100, 200)
except SoldeInsuffisant as e:
print(f"Refus : {e}")
```

4.4 Context managers (with)

L'instruction **with** garantit que les ressources sont libérées même en cas d'erreur. On peut créer ses propres context managers avec **__enter__** / **__exit__** ou avec **@contextmanager**.

```
# Lecture de fichier — fermeture automatique
with open("data.txt", "r", encoding="utf-8") as f:
contenu = f.read() # f.close() appelé automatiquement

# Context manager personnalisé avec contextlib
from contextlib import contextmanager

@contextmanager
def chrono(label):
import time
debut = time.time()
try:
yield
finally:
print(f"{label} : {time.time()-debut:.3f}s")

with chrono("Traitement"):
[x**2 for x in range(500_000)]
```

05 Modules et bonnes pratiques

Import, packages, PEP 8, type hints

5.1 Imports et modules

```
# Import complet
import math
print(math.sqrt(16)) # 4.0

# Import sélectif
from math import pi, cos
print(pi) # 3.14159...

# Import avec alias
import numpy as np
import pandas as pd

# Import conditionnel (évite les erreurs si absent)
try:
import ujson as json
except ImportError:
import json
```

5.2 Type hints (Python 3.5+)

Les **annotations de types** documentent les types attendus sans les imposer à l'exécution. Les outils comme mypy ou Pyright les utilisent pour détecter des erreurs statiquement.

```
from typing import Optional, Union, List, Dict

def saluer(nom: str, fois: int = 1) -> str:
return (f"Bonjour {nom}! " * fois).strip()

def diviser(a: float, b: float) -> Optional[float]:
return a / b if b != 0 else None

def traiter(items: List[Union[int, str]]) -> Dict[str, int]:
return {str(item): i for i, item in enumerate(items)}

# Python 3.10+ : syntaxe simplifiée
def nouvelle(x: int | None = None) -> list[int]:
return [x] if x else []
```

5.3 Bonnes pratiques PEP 8

Convention	Usage
`snake_case`	Noms de variables et fonctions
`PascalCase`	Noms de classes

Convention	Usage
`UPPER_CASE`	Constantes
`_privé`	Convention : attribut privé
`4 espaces`	Indentation (jamais des tabulations)
`79 caractères max`	Longueur max d'une ligne
`2 lignes vides`	Entre les fonctions et classes top-level
`Docstrings`	Toutes les fonctions/classes publiques

Astuce : *Utilise un linter (flake8, ruff) et un formateur (black, autopep8) dans ton éditeur pour respecter PEP 8 automatiquement.*